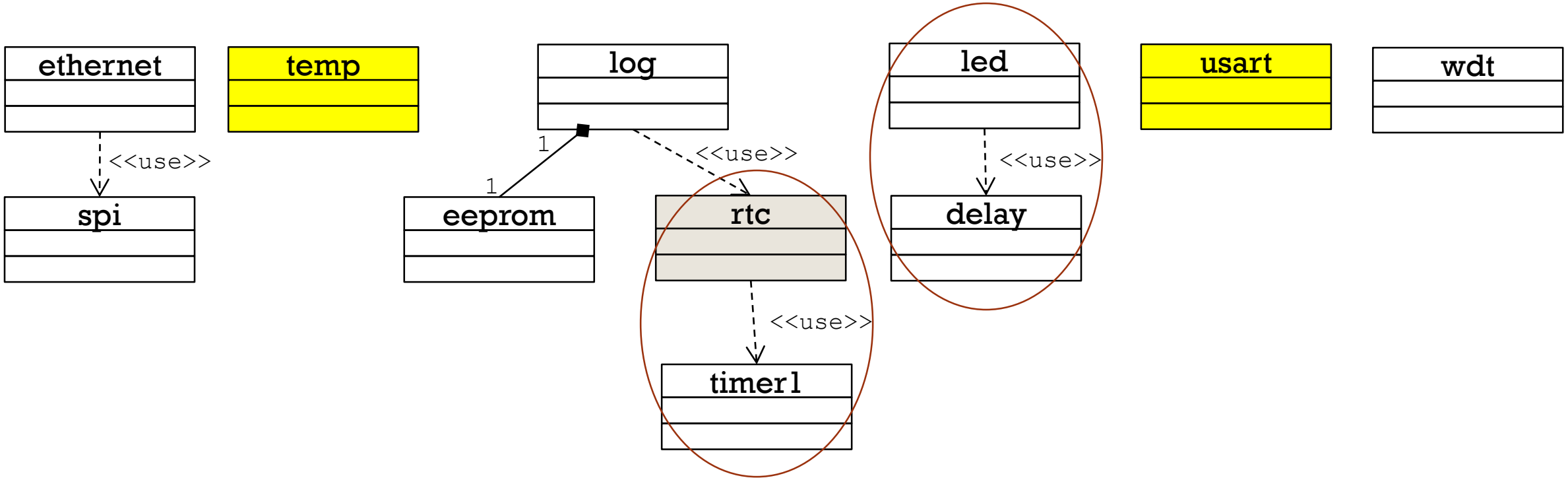
SER486: Embedded C Programming

Project 1
Design Specification

Arizona State University

PARTIAL 'CLASS' VIEW OF IIOT PROJECT



This project focuses on the circled classes.

DELAY CLASS

The delay class implements two instances of a delay timer which both utilize the timer0 hardware with a resolution of 1ms. Each time timer0 sends an interrupt, the delay counter for each delay instance is incremented.

- Internal State
  - **count[]** – the number of milliseconds elapsed since the delay started. Updated asynchronously by timer hardware (volatile)
  - **limit[]** – the number of milliseconds to delay.
  - **initialized** – 1 if the delay class has been initialized, 0 otherwise. (implement as a static variable within set().
- Member Functions
  - **void init()** – initialize the timer and the interrupt – called automatically the first time set() is called.
  - **unsigned int get(unsigned int num)** – returns the current value for count for the specified instance number. (guarantees atomic operation)
  - **void set(unsigned int num, unsigned int time)** – set the amount of time to delay (limit) and clear the elapsed time (count) for the specified instance number. (guarantees atomic operation)
  - **unsigned int isdone(unsigned int num)** – returns 1 if the delay count matches the limit for the specified instance, otherwise returns 0
  - **void __vector_14()** – invoked by 1ms timer interrupt. Increments the delay count by 1 (to be no greater than the limit)

delay
-count[] unsigned int -limit[] unsigned int -initialized unsigned char - TCCR0A, TCCR0B,TIMSK0, OCR0A
-init() +get(num) +set(num,newlimit) +isdone(num) - __vector_14()

PROGRAMMING THE TICK RATE
(HOW OFTEN THE TIMER INTERRUPT OCCURS)

- Tick Rate =
  CPU_FREQENCY / PRESCALER_VALUE / (COMPARE_VALUE+1)

  CPU_FREQUENCY = 16,000,000 Hz
  PRESCALER_VALUE = {1, 8, 64, 256, 1024} (selectable by software)
  COMPARE_VALUE = {0x00 : 0xFF} (for 8-bit timers)
  {0x00 : 0xFFFF} (for 16-bit timer)

PROCESS FOR SELECTING PRESCALER AND COMPARE VALUE

1. Find the largest prescaler value that will allow an integer compare value for the desired tick rate.
2. Calculate the compare value

For example, if you wanted a tick rate = 250 Hz (once every 4mSec)

16M/64 = 250,000Hz
Divisible by 250Hz => Yes
Compare = 250K / 250 -1= 999
This compare value is too big for an 8-bit compare register but it would be okay for 16-bits

16M / 256 = 62,500 Hz
Divisible by 250 => Yes
Compare = 62.5K/250-1=249

Best Solution

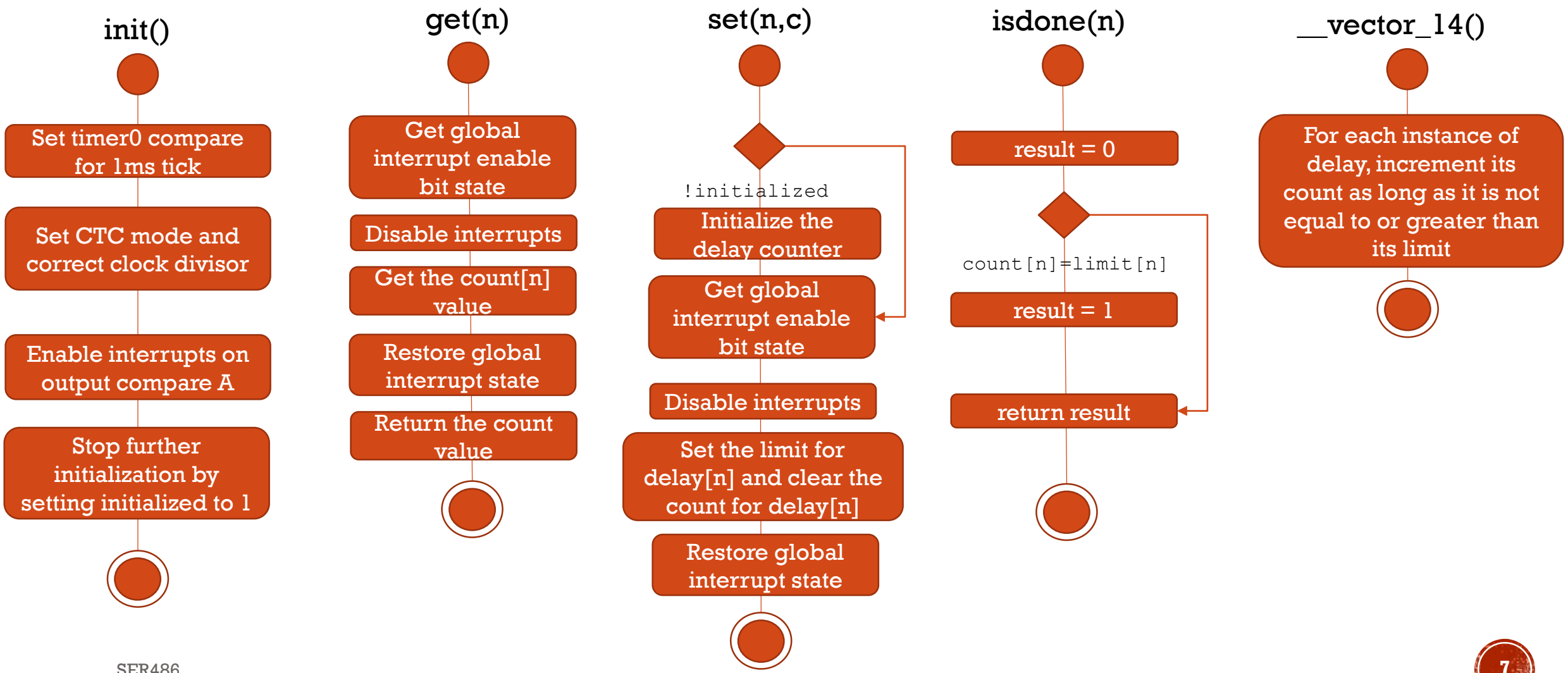
16M/1024 = 15625Hz
Divisible by 250Hz => No
This prescaler will not allow for precise timing due to rounding

NOTE: A 1mSecond tick period is a tick rate of 1,000Hz. A 1 second tick period is a tick rate of 1Hz

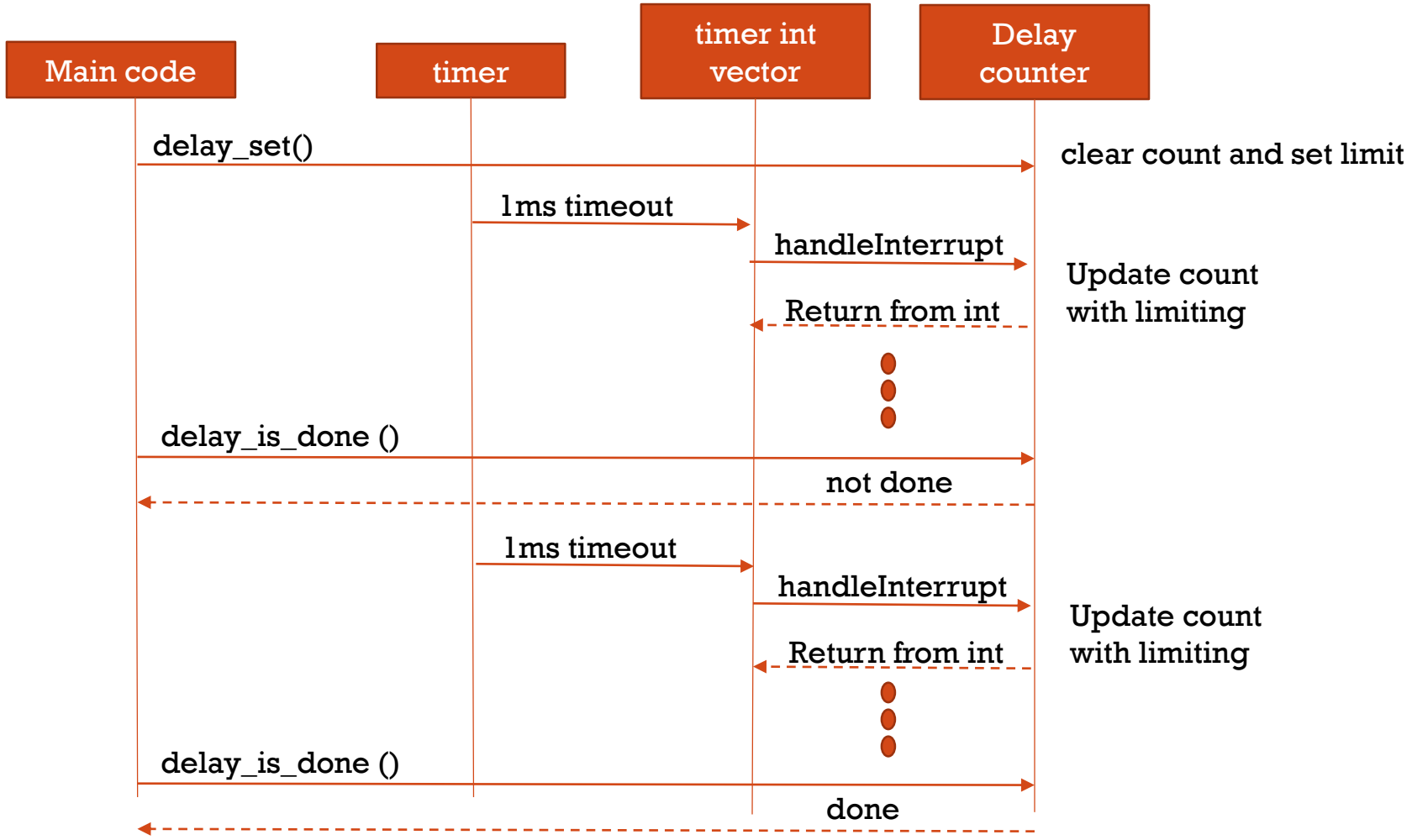
REGISTERS OF INTEREST

- 8 Bit Timers
  - TCCRxA and TCCRxB (control registers to set mode and clock divisor)
  - TIMSKx contains bit to enable interrupts from the timer
  - OCRxA – write this to set the compare value for CTC mode.
  - SREG – the ATMEGA status register (holds the value of the global interrupt enable)
- 16 Bit Timer
  - TCCR1A, TCCR1B, and TCCR1C (control registers to set mode and clock divisor)
  - TIMSK1 contains bit to enable interrupts from the timer
  - OCR1A (16-bits) – write this to set the compare value for CTC mode.
  - SREG – the ATMEGA status register (holds the value of the global interrupt enable)

DELAY METHODS



EXAMPLE: INTERRUPT HANDLER FOR DELAY



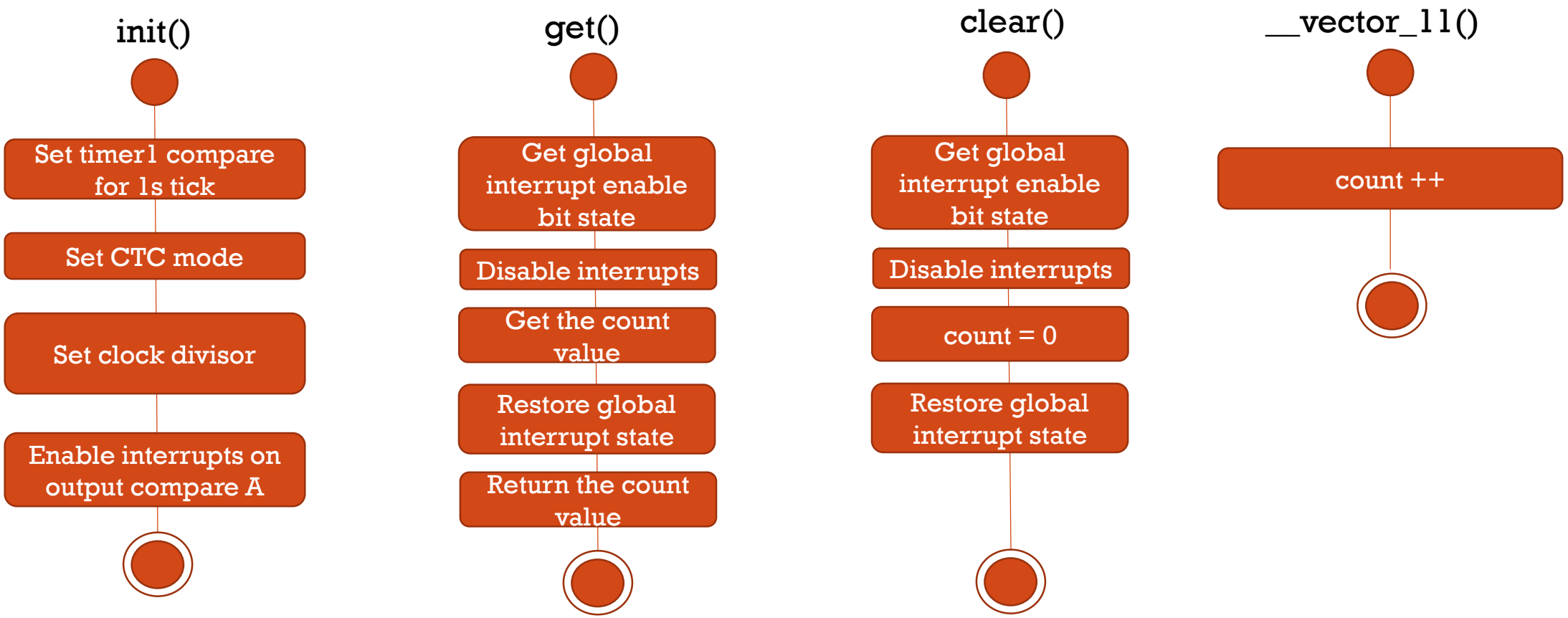
TIMER1 CLASS

The timer1 class utilizes the 16-bit counter/timer hardware to implement a free-running counter with a 1second update rate. The counter is implemented in software. Each 1 second, timer1 sends an interrupt and the counter is incremented.

- Internal State
  - **count** – the number of seconds elapsed since the timer started. Updated asynchronously by timer hardware (volatile)
- Member Functions
  - **void init()** – initialize the timer and the interrupt.
  - **unsigned long get()** – returns the current value for count. (guarantees atomic operation)
  - **void clear()** – clears the count value to 0 (guarantees atomic operation).
  - **void __vector_11()** – invoked by 1s timer interrupt. Increments the count by .

timer1
-count unsigned long -TCCR1A, TCCR1B, TCCR1C -TIMSK1, OCR1A
+init() +get() +clear() - __vector_11()

TIMER1 METHODS



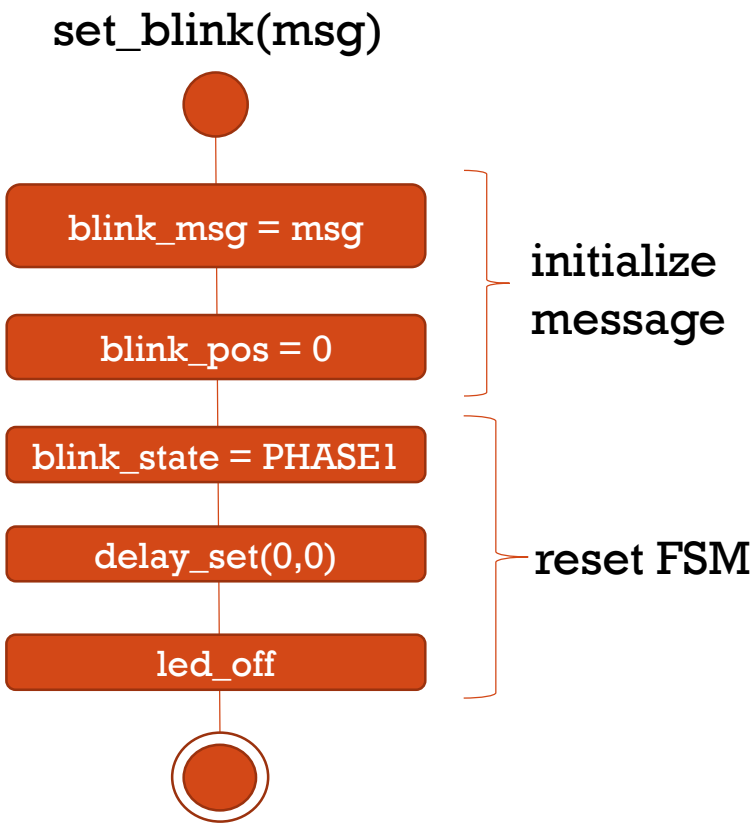
REVISED LED CLASS

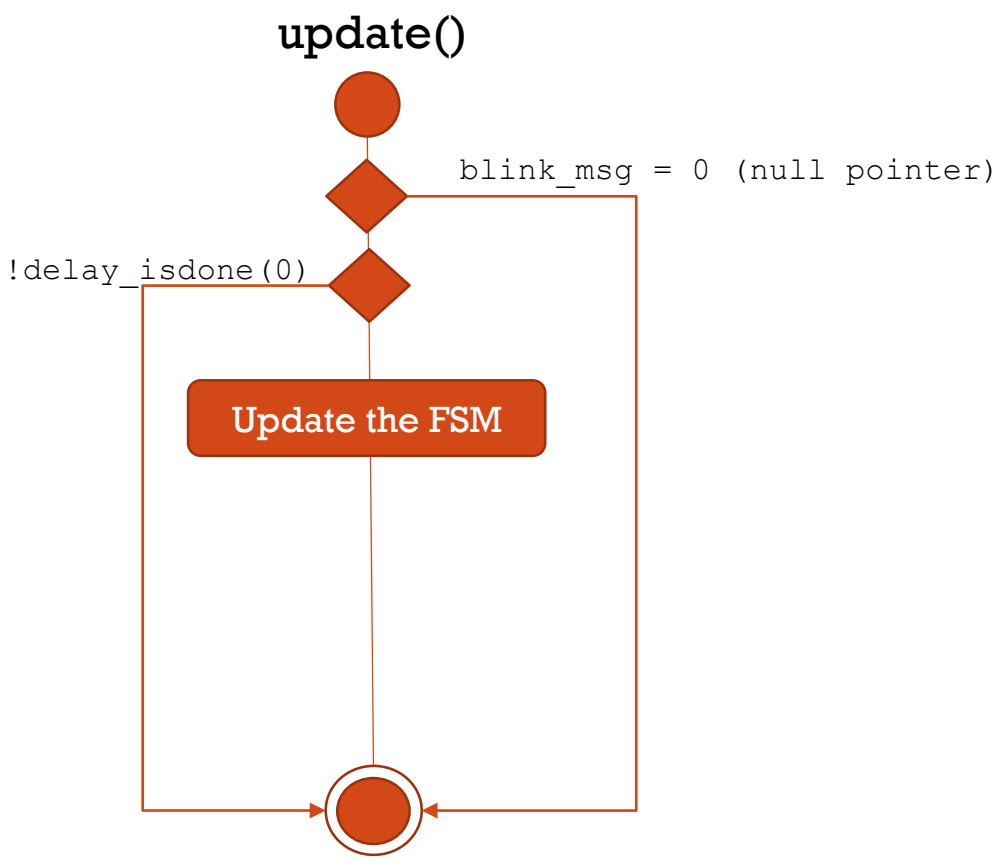
This class implements basic LED control as well as the ability to blink Morse code patterns on the LED.

- Data Members
  - PORTB, DDRB – (volatile) pin direction and output value.
  - **blink_msg** – a pointer to the Morse code string to be blinked.
  - **blink_pos** – the position in the blink message that is being blinked.
  - **blink_state** – the state of the led blink FSM.
- Member Functions
  - **void init()** – initialize the LED pin direction and turn off LED. (Instructor provided within library code)
  - **void on() / void off()** – turn on or off the LED. (Instructor provided within library code)
  - **char is_on()** – returns 1 if the LED is on, 0 if off. (Instructor provided within library code)
  - **void set_blink()** – sets the blink pattern to a new message, clears the blink FSM.
  - **void update()** – called as part of the main control loop to update the state of the LED FSM.

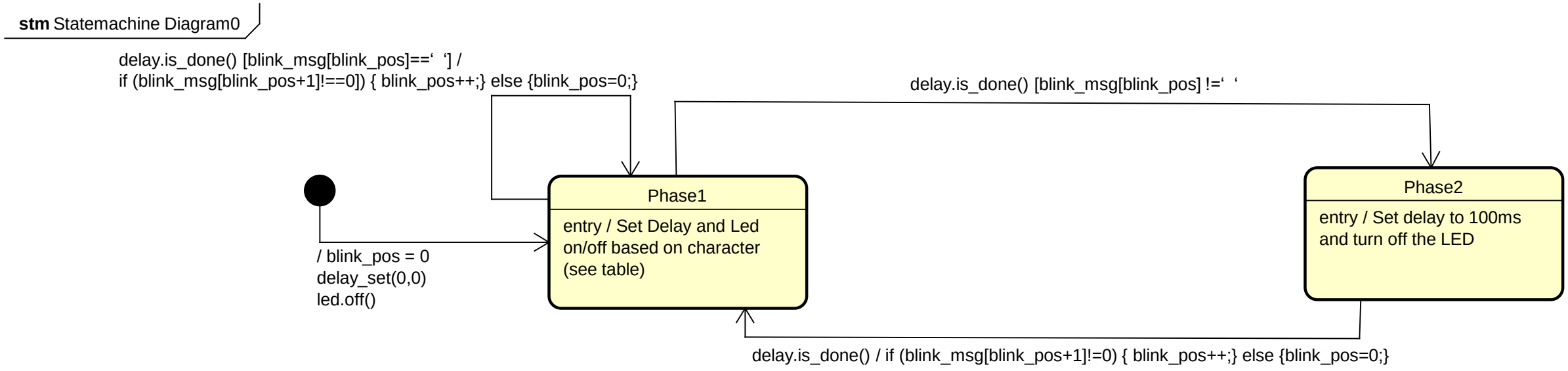
led
-PORTB pin 1 -DDRB pin 1 -blink_msg char* -blink_pos unsigned int -blink_state unsigned char
+init() +on() +off() +is_on() +set_blink() +update()

STUDENT PROVIDED LED METHODS





LED FINITE STATE MACHINE (FSM)



char	delay	led
' '	1000ms	off
'-'	750ms	on
'.'	250ms	on
Other	0ms	off

delay instance 0 is used for all LED timing functions

RTC CLASS (INSTRUCTOR PROVIDED)

This class uses the tick count value from Timer1 to implement a real-time (date/time) clock for data logging purposes.

- Data Members
  - **base** – time set by user expressed as seconds since 1/1/2000 0:0:0
  - **buf** – a buffer used for text date/time string creation
- Member Functions
  - **void init()** – initialize Timer1
  - **char * get_date_string()** - return the current date/time expressed in MM/DD/YYYY HH:MM:SS format.
  - **void set_by_datestr(char* datestr)** – set the current date/time from stiring expressed in MM/DD/YYYY HH:MM:SS format.
  - **unsigned long get_date()** – get the date time number expressed as seconds since 1/1/2000 00:00:00
  - **char * num2datestr(unsigned long datenum)** - return the a string representation of a given date number.

rtc
-base unsigned long -buf char*
+init() +get_date_string() +set_by_date_string() +get_date() +num2datestr()

/* BrainFuck.c
* BrainFuck Interpreter */
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
int main(int argc, char **argv)
{
int memory[32768], space[32768];
int fd, len, buf, spointer, mpointer;
int kl = 0;
if (argc <= 1)
{
printf("Usage: %s <brainfuck file>\n", argv[0]);
exit(1);
}
/* Open brainfuck file */
if ((fd = open(argv[1], O_RDONLY)) < 0) exit(1);
/* Read the file byte by byte */
len = spointer = 1;
while (len > 0)
{
len = read(fd, &buf, 1);
space[spointer] = buf;
spointer++;
}
close(fd);
for (mpointer = 0; mpointer < 32768; mpointer++) memory[mpointer] = 0;
len = spointer;
spointer = mpointer = 0;
for (spointer = 0; spointer < len; spointer++)
{
switch(space[spointer])
{
// Incrrment pointer value
case '+':
memory[mpointer]++;
break;
// Decrrment pointer value
case '-':
memory[mpointer]--;
break;
// Incrrment pointer
case '>':
mpointer++;
break;
// Decrrment pointer
case '<':
mpointer--;
break;
// Print current pointer value
case '.':
putchar(memory[mpointer]);
break;
// Read value and store in current pointer
case ',':
memory[mpointer] = getchar();
break;
// Start loop
case '[':
if (memory[mpointer] == 0)
{
/* Find matching] */
spointer++;
while (kl > 0 || space[spointer] != ']')
{
if (space[spointer] == '[') kl++;
if (space[spointer] == ']') kl--;
// Go in right direction
spointer++;
}
}
break;
// End Loop
case ']':
if (memory[mpointer] != 0)
{
/* Find matching [*/
spointer--;
while (kl > 0 || space[spointer] != '[')
{
if (space[spointer] == '[') kl--;
if (space[spointer] == ']') kl++;
// Go left
spointer--;
}
spointer--;
}
break;
}
}
putchar('\n');
return (0);
}

[END]